

KasparAI

Checkout Intelligence Agent

TRACK 2 · TEAM SEVEN UP

KasparAI

Checkout Intelligence Agent

TECHNICAL DOCUMENTATION

Full-Stack Architecture, API Design & AI Integration

Section 2

Technical Overview

KasparAI is a full-stack, real-time checkout recovery system. The frontend is a React 19 single-page application embedded in a simulated Shopify-compatible storefront. The backend is a Node.js/Express 5 REST API connected to MongoDB for session and event persistence. The AI layer uses OpenAI's GPT-4o-mini for hesitation classification and conversational recovery, with a complete rule-based fallback engine for environments without an API key.

Layer	Primary Technology	Version	Role
Frontend	React + Vite	19 / 8	Storefront UI, checkout form, KasparAI chat widget, behavioral tracking
Styling	Tailwind CSS + Framer Motion	4 / 12	Responsive layout, animation, micro-interactions
Backend	Node.js + Express	18+ / 5	REST API, session management, friction scoring, AI orchestration
Database	MongoDB + Mongoose	Atlas / 9	Session documents, event log collection, analytics aggregation
AI	OpenAI GPT-4o-mini	gpt-4o-mini	Hesitation classification, conversational recovery, dynamic responses
Shopify	Admin GraphQL API	2024-01	Live product catalog (optional; mock catalog if unconfigured)
HTTP Client	Axios	1.15	Frontend-to-backend API calls
Routing	React Router DOM	7	Client-side navigation between /, /store, /checkout, /dashboard

Section 3

System Architecture

The system follows a three-tier architecture. The presentation tier is the React SPA. The application tier is the Express API server. The data tier is MongoDB. AI services are consumed from the application tier via the OpenAI SDK. Shopify integration is also handled at the application tier via GraphQL HTTP calls.

Component	Responsibility	Communication Method
React SPA (Frontend)	Renders storefront, cart, checkout form, and KasparAI chat widget	HTTP/REST to Express API via Axios
useCheckoutTracker Hook	Tracks behavioral events; posts to /api/events on each signal	Axios POST on every event
CartContext	Global cart state; syncs to backend on every cart update	Axios POST to /api/sessions
Express API Server	Handles all business logic: session management, friction scoring, AI orchestration	REST endpoints; JSON request/response
analyzeHesitation()	Calls GPT-4o-mini or falls back to rule engine to classify hesitation reason	OpenAI SDK (HTTP)
runRecoveryConversation()	Multi-turn chat engine using three specialized agents + GPT-4o-mini	OpenAI SDK + in-memory agent logic
MongoDB (Sessions)	Persists session state: cart, friction score, intervention status, conversation history	Mongoose ODM
MongoDB (EventLog)	Append-only log of every behavioral event per session	Mongoose ODM
Shopify GraphQL API	Fetches live product catalog when credentials are present	node-fetch HTTP POST
Mock Catalog	30 hardcoded products across 7 categories; activated automatically without Shopify credentials	In-memory array

Section 4

Tech Stack

4.1 Frontend

Package	Version	Purpose
react	^19.2.5	Core UI rendering library
react-dom	^19.2.5	DOM rendering for React
vite	^8.0.10	Build tool and HMR dev server
react-router-dom	^7.14.2	Client-side routing (/ , /store, /checkout, /dashboard)
tailwindcss	^4.2.4	Utility-first CSS framework
framer-motion	^12.38.0	Animation for chat widget, cart transitions, page elements
axios	^1.15.2	HTTP client for API calls to Express backend
lucide-react	^1.11.0	Icon library for UI components
@stripe/stripe-js	^9.3.1	Stripe payment UI prototype (UI only, no real charges)

4.2 Backend

Package	Version	Purpose
express	^5.2.1	HTTP server and REST API routing
mongoose	^9.5.0	MongoDB ODM for schema definition and querying
openai	^6.34.0	OpenAI SDK for GPT-4o-mini API calls
@shopify/shopify-api	^13.0.0	Shopify API utilities (GraphQL client)
cors	^2.8.6	Cross-origin resource sharing middleware
dotenv	^17.4.2	Environment variable loading from .env file
nodemon	^3.1.14	Auto-restart on file changes in development (devDependency)

4.3 Database, APIs, and AI Integration

Technology	Type	Details
MongoDB	Database	NoSQL document store; Atlas or local; two collections: sessions, eventlogs
Shopify Admin GraphQL API	External API	Version 2024-01; fetches products with title, description, images, variants, price
OpenAI Chat Completions	AI API	Model: gpt-4o-mini; max_tokens: 300 for chat, unlimited for classification; response_format: json_object for classifier

Section 5

Frontend Architecture

5.1 Folder Structure

```
frontend/src/

■■■ App.jsx # Root component, router setup
■■■ main.jsx # ReactDOM render entry point
■■■ index.css # Global CSS resets
■■■ App.css # App-level styles
■■■ context/
■ ■■■ CartContext.jsx # Global cart state + sessionId management
■ ■■■ AuthContext.jsx # Authentication state (prototype)
■■■ hooks/
■ ■■■ useCheckoutTracker.js # Behavioral event tracking hook
■■■ components/
■ ■■■ Navbar.jsx # Top-level navigation bar
■ ■■■ layout/Navbar.jsx # Layout-specific Navbar variant
■■■ pages/
■■■ Home.jsx # Landing page
■■■ Store.jsx # Product catalog page
■■■ Checkout.jsx # Main checkout + KasparAI widget
■■■ Dashboard.jsx # Merchant analytics dashboard
■■■ Demo.jsx # Feature demonstration page
■■■ Login.jsx # Login form (prototype)
■■■ Signup.jsx # Signup form (prototype)
```

5.2 State Management

State Layer	Technology	What It Holds
Global cart state	React Context (CartContext)	cart items, sessionId, addToCart, updateQuantity, clearCart, getCartTotal
Global auth state	React Context (AuthContext)	user object, login/logout handlers (prototype)
Checkout form state	useState (local to Checkout.jsx)	contact info, shipping address, payment fields, coupon code, selected shipping method
Chat state	useState (local to Checkout.jsx)	chatOpen, messages array, loading flag, resolved flag, offerType

State Layer	Technology	What It Holds
Tracking state	useRef (in useCheckoutTracker)	idle timers, section dwell timers, proactive tick counter

Section 6

Backend Architecture

6.1 Server Setup

The backend is a single-file Express 5 server (server.js) using ES modules (type: module in package.json). It connects to MongoDB via Mongoose on startup and registers seven REST API routes. CORS is enabled globally to allow the React dev server (localhost:5173) to communicate with the API (localhost:5000).

6.2 Route Structure

Method	Endpoint	Controller Logic	Key Operations
GET	/api/products	fetchShopifyProducts()	Returns live Shopify catalog or 30-item mock array
POST	/api/sessions	Session.findOne + save()	Upserts session by sessionId; updates cart and total
POST	/api/events	EventLog.save + friction logic	Logs event; increments friction score; triggers classification at score >= 50
GET	/api/intervention/:sessionId	Session.findOne	Returns { triggered, type, frictionScore } for frontend polling
POST	/api/chat	runRecoveryConversation()	Accepts user message; returns AI reply; updates conversation history
POST	/api/sessions/:sessionId/complete	Session.save	Sets checkoutStage = 'completed'; used to mark recovered sessions
GET	/api/analytics	Session.aggregate	Returns totalSessions, recovered, interventions, byType breakdown

6.3 Agents Folder

Agent File	Class	Exported Methods
agents/CheckoutAgent.js	CheckoutAgent	analyzeHesitation(session, events) → string null; persuade(session) → string
agents/PricingAgent.js	PricingAgent	generateOffer(session) → { type, code, message } null; validateCoupon(code) → { discount, type } null
agents/ShoppingAgent.js	ShoppingAgent	recommend(session, history) → string; search(query) → Product[]

Section 7

Database Design

7.1 Session Collection Schema

Field	Type	Default	Description
sessionId	String (unique, required)	—	Frontend-generated ID (sess_XXXXXXX); persistent in localStorage
checkoutStage	String	'cart'	Lifecycle stage: cart checkout payment completed
cartItems	Array	[]	Array of { product, variant, quantity } objects synced from CartContext
cartTotal	Number	0	Computed cart total in dollars; updated on every cart sync
frictionScore	Number	0	Cumulative behavioral friction score (0–100); reset to 25 on low-confidence classification
interventionTriggered	Boolean	false	Set to true when hesitation classification succeeds; prevents duplicate interventions
interventionType	String	null	Classified hesitation reason (e.g., 'price hesitation')
conversationHistory	Array	[]	Array of { role: 'user' 'assistant', content: string } for multi-turn chat context
recoveredAt	Date	null	Timestamp of successful checkout completion after intervention
createdAt	Date	Date.now	Session creation timestamp
updatedAt	Date	Date.now	Last modification timestamp

7.2 EventLog Collection Schema

Field	Type	Description
sessionId	String (required)	Links event to its parent session document
eventType	String (required)	One of: idle_checkout, cart_modified, shipping_method_changed, address_edited, coupon_failed, exit_intent, price_hover, payment_error, section_dwell, payment_field_focused, product_hover, quantity_changed, remove_item_attempted
metadata	Mixed (JSON)	Context object: { section, method, field, code, item, dwell_ms, etc. } — varies by event type
timestamp	Date	Auto-generated at event creation; used for chronological event replay in classification

Section 8

Shopify Integration

Shopify integration is optional and gracefully degraded. When SHOPIFY_STORE_DOMAIN and SHOPIFY_ADMIN_ACCESS_TOKEN are present in the environment, the system fetches live product data from the Shopify Admin GraphQL API. When credentials are absent, a 30-item mock catalog is returned instead, allowing full demo functionality without a Shopify account.

Integration Point	Details
API Version	Shopify Admin API 2024-01
Endpoint	https://{domain}/admin/api/2024-01/graphql.json
Auth Header	X-Shopify-Access-Token: {token}
Query Scope	products(first: 30) — fetches id, title, description, images(first:1), variants(first:10) with id, title, price, inventoryQuantity
Product Mapping	mapShopifyProducts() normalizes Shopify GraphQL nodes into { id, title, description, category, image, variants: [{ id, title, price }] }
Category Assignment	Category is inferred from product title keywords — not a Shopify native field — enabling catalog browsing by category in the frontend
Fallback Trigger	fetchShopifyProducts() returns mockProducts when credentials are missing or when the GraphQL call throws any error
Mock Catalog	30 hardcoded products across 7 categories: Electronics, Fashion, Fitness, Kitchen, Home Decor, Books, Beauty — each with 2–3 variants and realistic prices

Section 9

OpenAI Integration

9.1 Hesitation Classifier Prompt Engineering

The classifier receives a structured JSON payload containing cart total, item count, friction score, and the full event log. It is instructed to output only valid JSON with three fields: reason, confidence, and explanation. The response_format: json_object parameter ensures well-formed JSON output.

```
System Prompt:

You are a checkout friction classifier for an e-commerce checkout agent.

Based on the following user session behavior, determine the PRIMARY reason
this user is hesitating. Output strictly valid JSON only:

{ "reason": "price hesitation" | "shipping confusion" | "coupon confusion"
  | "exit intent" | "payment error" | "trust gap" | "other",
  "confidence": ,
  "explanation": "" }
```

9.2 Recovery Conversation Prompt Engineering

The recovery conversation system prompt includes: the hesitation type, cart summary (items + total), friction score, available discount codes, and instructions to emit [RESOLVED: offer_type] when the shopper accepts an offer. The specialized agent context (PricingAgent offer, ShoppingAgent recommendation, or CheckoutAgent persuasion) is appended to the system prompt dynamically.

AI Call	Model	Max Tokens	Temperature	Response Format
analyzeHesitation()	gpt-4o-mini	1000 (SDK default)	Default (1.0)	json_object
runRecoveryConversation()	gpt-4o-mini	300	Default (1.0)	text

9.3 Context Handling and Conversation Memory

Each call to POST /api/chat retrieves the session's conversationHistory array from MongoDB and includes it in the messages array sent to the OpenAI API. This gives the model full awareness of the conversation so far. After each turn, the updated history (including the new user message and assistant reply) is saved back to MongoDB, enabling true multi-turn statefulness across multiple HTTP requests without requiring a persistent WebSocket connection.

Section 10

Feature-Wise Technical Implementation

Feature	Frontend Implementation	Backend Implementation
Behavioral Event Tracking	useCheckoutTracker hook: registers mousemove/keypress/scroll/click listeners; uses setTimeout/setInterval for idle detection; uses Date.now() for section dwell timing	POST /api/events: validates sessionId and eventType; saves EventLog document; updates session friction score
Friction Scoring	N/A — computed server-side	frictionMap object in server.js maps eventType → score increment; score capped at 100 using Math.min; stored in session.frictionScore
Hesitation Classification	Frontend polls GET /api/intervention/:sessionId every 2 seconds after checkout page loads	analyzeHesitation() in utils/analyzer.js: builds summaryData JSON; calls GPT-4o-mini with json_object response_format; validates confidence > 0.55; returns reason string or null
KasparAI Chat Widget	Conditional rendering in Checkout.jsx: chatOpen state controls AnimatePresence animation; TRIGGER_MESSAGES object maps hesitation type to opening message; PROACTIVE_SEQUENCE sends timed messages every 10 seconds	POST /api/chat: calls runRecoveryConversation(); saves updated history; sets recoveredAt if resolved
Offer Generation	Displays offer code in formatted chat message; OFFER_LABELS maps offerType to display string	PricingAgent.generateOffer() checks cartTotal threshold (> \$500 → WELCOME15, else SAVE10); PricingAgent.validateCoupon() returns discount object
Session Recovery	sessionId read from localStorage on every page load; CartContext reinitializes from localStorage 'cart' key	POST /api/sessions upserts session by sessionId — creates new or updates existing
Merchant Dashboard	Dashboard.jsx renders bar chart (Recharts or custom SVG), recovery metrics, intervention breakdown	GET /api/analytics: uses MongoDB aggregate pipeline to group by interventionType and count recoveries

Section 11

Functions and Modules

Function / Module	Location	Parameters	Returns	Purpose
fetchShopifyProducts()	utils/shopify.js	none	Product[]	Fetches Shopify catalog or returns mockProducts
analyzeHesitation()	utils/analyzer.js	session, eventLogs[]	string null	GPT-4o-mini or rule-based hesitation classifier
runFallbackRules()	utils/analyzer.js	session, eventLogs[]	string null	Deterministic event-count classification
runRecoveryConversation()	utils/conversation.js	session, history[], userMessage	{ reply, resolved, offerType, updatedHistory }	Multi-turn AI or fallback conversation engine
runFallbackConversation()	utils/conversation.js	session, history, msg, context	same as above	Rule-based conversation when OpenAI is unavailable
buildSystemPrompt()	utils/conversation.js	session, specializedContext	string	Constructs GPT system prompt with session context
detectResolution()	utils/conversation.js	reply	boolean	Checks if reply contains [RESOLVED:...]
detectOfferType()	utils/conversation.js	reply	string	Extracts offer type from [RESOLVED: offer_type]
CheckoutAgent.analyzeHesitation()	agents/CheckoutAgent.js	session, events[]	string null	Pattern-based checkout hesitation analysis
CheckoutAgent.persuade()	agents/CheckoutAgent.js	session	string	Returns random persuasion message
PricingAgent.generateOffer()	agents/PricingAgent.js	session	offer object null	Cart-value-aware discount offer generator
PricingAgent.validateCoupon()	agents/PricingAgent.js	code	{ discount, type } null	Validates coupon against hardcoded dictionary

Function / Module	Location	Parameters	Returns	Purpose
ShoppingAgent.recommend())	agents/ShoppingAgent.js	session, history	string	Category-based product recommendation message
ShoppingAgent.search()	agents/ShoppingAgent.js	query	Product[]	Fuzzy-search products by title/description/category
useCheckoutTracker()	hooks/useCheckoutTracker.js	stage, { onProactiveTick }	tracker methods object	Behavioral tracking hook for checkout page
CartProvider	context/CartContext.jsx	{ children }	JSX	Global cart context provider with localStorage persistence

Section 12

AI vs. Deterministic Logic

Decision Point	AI (GPT-4o-mini)	Deterministic / Rule-Based	Trigger for AI
Hesitation classification	Primary path when API key present	runFallbackRules() — event count thresholds	OPENAI_API_KEY env var is set and valid
Recovery conversation	Primary path — full conversational response generation	runFallbackConversation() — template strings by hesitation type	OPENAI_API_KEY env var is set and valid
Discount offer selection	N/A — fully deterministic	PricingAgent.generateOffer() — cartTotal threshold logic	Never — pricing is always deterministic
Coupon validation	N/A — fully deterministic	PricingAgent.validateCoupon() — dictionary lookup	Never — validation is always deterministic
Product recommendations	Optional enhancement in system prompt	ShoppingAgent.recommend() — array filter by category	When OpenAI conversation path is active
Opening message selection	N/A — template-based	TRIGGER_MESSAGES / OPENING_MESSAGES dictionary — key = hesitation type	Never — opening message is always templated

Section 13

Checkout Tracking System

The useCheckoutTracker hook is the behavioral intelligence layer of the frontend. It operates entirely through side effects and exposes a set of callable tracking functions to the Checkout component.

Tracking Mechanism	Implementation	Event Emitted
Idle detection	resetIdle() resets a 10-second setTimeout on any user interaction. After 10s of inactivity, setInterval fires idle_checkout every 10s	idle_checkout { stage, section }
Section dwell	enterSection(name) records Date.now(); on next enterSection() call, if elapsed > 8000ms, logs the previous section's dwell	section_dwell { section, dwell_ms }
Proactive tick	setInterval fires onProactiveTick(count) every 10s — independent of user activity; used to push proactive chat messages	n/a (callback, not logged to backend)
Exit intent	trackExitIntent() called from mouseleave on document; also called from trackRemoveAttempt()	exit_intent { stage } or { reason: 'remove_item_attempt' }
Shipping change	trackShippingChange(method) called from shipping radio onChange handler in Checkout.jsx	shipping_method_changed { method }
Coupon failure	trackCouponFail(code) called when coupon validation logic detects invalid code	coupon_failed { code }
Payment focus	trackPaymentFocus(field) called on onFocus of card number, CVV, expiry fields	payment_field_focused { field }
Quantity change	trackQuantityChange(item, oldQty, newQty) called from cart quantity stepper	quantity_changed { item, oldQty, newQty }

Section 14

Friction Detection Engine

The friction engine is implemented in the POST /api/events route handler in server.js. It maintains a cumulative score per session and triggers AI classification when the threshold is reached.

Event Type	Score Increment	Rationale
exit_intent	+30	Highest signal — user is actively moving to leave
coupon_failed	+25	Strong signal of value-seeking friction
idle_checkout	+20	Sustained inactivity indicates confusion or hesitation
payment_error	+20	Payment failure is a high-friction moment
shipping_method_changed	+15	Repeated changes indicate shipping uncertainty
price_hover	+15	Cursor lingering on price indicates cost sensitivity
address_edited	+10	Minor friction signal — could be correction
cart_modified	+10	Quantity or item changes signal indecision
(any other event)	+5	Low-signal catch-all for unclassified events

Threshold Condition	Action
frictionScore >= 50 AND checkoutStage != 'completed' AND NOT interventionTriggered	Call analyzeHesitation(); if reason returned, set interventionTriggered=true and save reason
analyzeHesitation() returns null (low confidence)	Reset frictionScore to 25; prevents repeated low-confidence triggers in the same session
interventionTriggered already true	Return { status: 'already_intervened' } immediately; no additional classification

Section 15

Session Management

Aspect	Implementation
Session ID generation	Frontend generates 'sess_' + Math.random().toString(36).substr(2,9) on first load; stored in localStorage under key 'sessionId'
Session persistence across tabs	localStorage ensures the same sessionId is used if the user opens the store in the same browser
Session creation	POST /api/sessions with sessionId creates a new MongoDB Session document if one does not exist
Session update	POST /api/sessions with existing sessionId updates cartItems and cartTotal without resetting other fields
Conversation history storage	conversationHistory array in the Session document is appended on every POST /api/chat call and re-sent to OpenAI on the next turn
Session completion	POST /api/sessions/:sessionId/complete sets checkoutStage = 'completed'; prevents further friction scoring and intervention
Analytics query	GET /api/analytics uses MongoDB's countDocuments() for totals and aggregate() with \$group for type breakdown

Section 16

Error Handling

Error Scenario	Location	Handling Strategy
OpenAI API call fails (network error, rate limit, auth failure)	utils/analyzer.js, utils/conversation.js	try/catch around client.chat.completions.create(); falls back to runFallbackRules() or runFallbackConversation()
MongoDB connection fails on startup	server.js mongoose.connect()	Console error logged; process continues; subsequent DB operations will fail with caught errors
Session not found (404)	Any route using Session.findOne()	Returns res.status(404).json({ error: 'Session not found' })
Missing required fields in request body	POST /api/events, POST /api/chat, POST /api/sessions	Returns res.status(400).json({ error: '...' }) with descriptive message
Shopify API fetch fails	utils/shopify.js fetchShopifyProducts()	catch block logs error and returns mockProducts; storefront always has product data
Frontend API call fails	CartContext, useCheckoutTracker, Checkout.jsx	Axios .catch() => {} silently ignores tracking failures; chat errors show retry prompt
JSON parse error on AI response	utils/analyzer.js classification parse	try/catch returns runFallbackRules() on JSON.parse failure

Section 17

Failure Recovery

Recovery Path	Trigger	Behavior
Rule-based hesitation classification	OpenAI API unavailable or OPENAI_API_KEY not set	runFallbackRules() counts event types and maps counts to hesitation categories deterministically
Rule-based recovery conversation	OpenAI API unavailable during chat	runFallbackConversation() returns pre-written response templates based on session.interventionType and userMessage keyword matching
Mock product catalog	Shopify credentials absent or Shopify API error	fetchShopifyProducts() returns 30 hardcoded products; all store functionality works normally
Friction score partial reset	Classification returns null (low confidence)	frictionScore reset to 25; allows future events to re-trigger classification without a false intervention
Frontend silent failure	Backend unreachable for event tracking	Tracking axios calls use .catch(() => {}) — tracking failures never interrupt the user experience
Cart state preservation	Page refresh or navigation	localStorage holds cart JSON and sessionId; CartContext restores state on mount

Section 18

Security Measures

■ Payment fields in the current implementation are UI prototypes. No real payment data is collected, stored, or processed. Stripe integration exists at the SDK import level only.

Security Area	Implementation
API key handling	OPENAI_API_KEY and SHOPIFY_ADMIN_ACCESS_TOKEN loaded from .env via dotenv; never exposed in frontend code or API responses
CORS configuration	cors() middleware in Express accepts all origins in development; should be scoped to specific domains in production
Input validation	sessionId and eventType are checked for presence in POST /api/events; malformed JSON rejected by Express middleware
MongoDB injection prevention	Mongoose ODM parameterizes all queries; Session.findOne({ sessionId }) uses safe object syntax
Environment separation	.env.template and .env.example committed; actual .env files in .gitignore
No authentication on API	Routes are unauthenticated in the hackathon demo; production would require API key or JWT middleware
No PII collection	SessionIds are random strings; no user emails, names, or payment data are stored in MongoDB

Section 19

API Design

Endpoint	Request Body / Params	Response Schema	HTTP Status Codes
GET /api/products	none	Product[] — [{ id, title, description, category, image, variants: [{ id, title, price }] }]	200, 500
POST /api/sessions	{ sessionId, cartItems[], cartTotal }	{ success: true, session: SessionDoc }	200, 400, 500
POST /api/events	{ sessionId, eventType, metadata? }	{ success: true, frictionScore? } or { intervention: string } or { status: 'already_intervened' }	200, 400, 404, 500
GET /api/intervention/:sessionId	:sessionId path param	{ triggered: bool, type: string null, frictionScore: number }	200, 404, 500
POST /api/chat	{ sessionId, userMessage }	{ reply: string, resolved: bool, offerType: string }	200, 400, 404, 500
POST /api/sessions/:sessionId/complete	:sessionId path param	{ success: true }	200, 404, 500
GET /api/analytics	none	{ totalSessions, recovered, interventions, byType: [{ _id: string, count: number }] }	200, 500

Section 20

Performance Optimizations

Optimization	Details
Product catalog caching	productsCache array in conversation.js avoids re-fetching Shopify products on every chat call; populated once on first request
Friction score cap	Math.min(..., 100) prevents unbounded score growth; score capped at 100 regardless of event volume
Polling instead of WebSockets	GET /api/intervention uses 2-second polling; avoids WebSocket connection overhead for the intervention signal which fires at most once per session
React useCallback for tracker	All tracker functions in useCheckoutTracker are wrapped in useCallback with [sessionId] dependency to prevent unnecessary re-renders
useRef for timers	Idle timers and section timers use useRef — not useState — ensuring timer mutations do not trigger component re-renders
Conversation history truncation (future)	conversationHistory grows unbounded in current implementation; production should truncate to last N turns to stay within GPT context window
Mongoose lean queries (future)	findOne().lean() would return plain JS objects instead of Mongoose documents for read-only operations, reducing memory overhead

Section 21

Scalability Considerations

Consideration	Current State	Production Path
Database	Single MongoDB instance (local or Atlas M0)	Atlas M10+ with replica sets; read replicas for analytics queries
Session store	MongoDB documents	Redis for high-frequency session reads; MongoDB for persistence
API server	Single Express process	Horizontally scaled behind a load balancer (e.g., AWS ALB); PM2 cluster mode
OpenAI rate limits	No rate limit handling beyond catch/fallback	Request queuing with bull/BullMQ; exponential backoff retry logic
Event volume	Synchronous event write per HTTP request	Event queue (e.g., Kafka or SQS) for high-traffic stores; batch writes to MongoDB
Conversation history	Full history per API call	Sliding window of last 10 turns to stay within GPT token limits at scale
Frontend	Vite dev server / static build	CDN-deployed static bundle (Vercel, Netlify, CloudFront)

Section 22

Limitations

Limitation	Impact	Resolution Path
Unresolved git merge conflicts in conversation.js and shopify.js	Backend will not start if conflict markers are not resolved before running npm run dev	Manually resolve by choosing the HEAD version (upstream branch contains agent imports; HEAD has OPENING_MESSAGES)
Payment is UI-only	No real transactions; checkout completion is simulated	Integrate Stripe Checkout Elements with real publishable/secret keys
Analytics dashboard uses mock data	Charts and recovery numbers are not wired to live /api/analytics endpoint	Connect Dashboard.jsx to GET /api/analytics and replace mock state with API data
No authentication	Any client with a valid sessionId can read/write any session	Add JWT middleware to all session-specific endpoints in production
conversationHistory grows unbounded	Very long sessions may exceed GPT context window limit	Implement sliding window: keep last 10 turns only
No request rate limiting	API endpoints are unprotected from abuse	Add express-rate-limit middleware; authenticate merchant API consumers
Shopify Storefront API not used	Product fetching uses Admin API (requires high-permission token)	Switch to Shopify Storefront API for read-only product access with a lower-privilege token

Section 23

Future Improvements

Improvement	Technical Approach	Priority
Resolve merge conflicts and stabilize backend	Choose authoritative branch; merge OPENING_MESSAGES and agent imports into single conversation.js	Critical
Wire dashboard to live analytics	Replace mock state in Dashboard.jsx with useEffect + axios GET /api/analytics	High
Stripe payment integration	Replace UI prototype fields with StripeElements; add /api/payment-intent backend route	High
WebSocket-based intervention push	Replace polling with Socket.io; server emits 'intervention' event on session update	Medium
Conversation history windowing	Slice conversationHistory to last 10 turns before sending to OpenAI	High
Rate limiting and auth	express-rate-limit + JWT middleware on session-specific routes	High
Shopify Storefront API migration	Swap Admin API for Storefront API; use storefront access token with read_products scope only	Medium
Merchant configuration API	Add /api/merchant/config endpoint for threshold and message customization per store	Medium
CI/CD pipeline	GitHub Actions: lint + test on PR; deploy backend to Railway/Render; frontend to Vercel on merge to main	Low

Section 24

Setup Instructions

Prerequisites

Requirement	Minimum Version	Purpose
Node.js	18.x	Runtime for both backend and frontend build tooling
npm	9.x	Package manager
MongoDB	6.x (local) or Atlas	Session and event persistence
OpenAI API Key	Optional	Enables GPT-4o-mini classification; falls back to rules without it
Shopify Store + Admin Token	Optional	Enables live product catalog; falls back to 30-item mock

Installation Steps

```
# 1. Clone the repository
git clone https://github.com/aadyasengar/checkout-intelligence-agent.git
cd checkout-intelligence-agent

# 2. Install backend dependencies
cd backend && npm install

# 3. Configure backend environment
cp .env.template .env
# Edit .env – set MONGODB_URI, optionally OPENAI_API_KEY and SHOPIFY creds

# 4. Resolve merge conflicts (required before starting)
# Open backend/utils/conversation.js and backend/utils/shopify.js
# Remove all <<<<<, =====, >>>>> conflict markers
# Keep the HEAD version (OPENING_MESSAGES + agent imports both needed)

# 5. Install frontend dependencies
cd ../frontend && npm install

# 6. Configure frontend environment
cp .env.example .env
# VITE_API_URL=http://localhost:5000/api (default; no change needed)

# 7. Start backend (Terminal 1)
cd backend && npm run dev

# 8. Start frontend (Terminal 2)
```

```
cd frontend && npm run dev  
  
# Open http://localhost:5173 in browser
```

Section 25

Environment Variables

Backend (.env)

Variable	Required	Default	Description
MONGODB_URI	Yes	mongodb://127.0.0.1:27017/checkout-recovery	MongoDB connection string; use Atlas URI for cloud deployment
PORT	No	5000	Port the Express server listens on
OPENAI_API_KEY	No	—	OpenAI API key; enables GPT-4o-mini; rule-based fallback activates if absent
SHOPIFY_STORE_DOMAIN	No	—	Shopify store domain (e.g. your-store.myshopify.com); mock catalog used if absent
SHOPIFY_ADMIN_ACCESS_TO KEN	No	—	Shopify Admin API access token (shpat_...); required alongside domain for live products

Frontend (.env)

Variable	Required	Default	Description
VITE_API_URL	No	http://localhost:5000/api	Base URL for all Axios API calls from the frontend; update for production deployment

Section 26

Deployment Instructions

Component	Recommended Platform	Deployment Steps
Backend (Node.js/Express)	Railway or Render	Connect GitHub repo; set root to /backend; add environment variables (MONGODB_URI, OPENAI_API_KEY, etc.); deploy automatically on push to main
Frontend (React/Vite)	Vercel or Netlify	Connect GitHub repo; set root to /frontend; build command: npm run build; output directory: dist; set VITE_API_URL to production backend URL
Database (MongoDB)	MongoDB Atlas	Create free M0 cluster; get connection string; add to backend MONGODB_URI; whitelist deployment IP or use 0.0.0.0/0 for demo

Section 27

Repository Structure

```
checkout-intelligence-agent/  
  
  ■■■ backend/  
  ■ ■■■ agents/  
    ■ ■ ■ CheckoutAgent.js # Hesitation analysis + persuasion  
    ■ ■ ■ PricingAgent.js # Discount generation + coupon validation  
    ■ ■ ■ ShoppingAgent.js # Product recommendations + catalog search  
  ■ ■■■ models/  
    ■ ■ ■ Session.js # Mongoose schema for checkout sessions  
    ■ ■ ■ EventLog.js # Mongoose schema for behavioral events  
  ■ ■■■ utils/  
    ■ ■ ■ analyzer.js # GPT-4o-mini hesitation classifier + fallback  
    ■ ■ ■ conversation.js # Multi-turn recovery conversation engine  
    ■ ■ ■ shopify.js # Shopify GraphQL client + 30-item mock catalog  
  ■ ■■■ server.js # Express app + all 7 API routes  
  ■ ■■■ package.json  
  ■ ■■■ .env.template  
  
  ■■■ frontend/  
  ■■■ src/  
    ■ ■■■ context/  
      ■ ■ ■ CartContext.jsx # Global cart state + sessionId  
      ■ ■ ■ AuthContext.jsx # Auth state (prototype)  
    ■ ■■■ hooks/  
      ■ ■ ■ useCheckoutTracker.js # Behavioral event tracking  
    ■ ■■■ components/  
      ■ ■ ■ layout/Navbar.jsx  
    ■ ■■■ pages/  
      ■ ■ ■ Home.jsx # Landing page  
      ■ ■ ■ Store.jsx # Product catalog  
      ■ ■ ■ Checkout.jsx # Checkout + KasparAI widget  
      ■ ■ ■ Dashboard.jsx # Merchant analytics  
      ■ ■ ■ Demo.jsx  
      ■ ■ ■ Login.jsx  
      ■ ■ ■ Signup.jsx  
    ■ ■■■ App.jsx # Router setup
```

```
■ ■■■ main.jsx # Entry point
■■■ package.json
■■■ vite.config.js
■■■ .env.example
```

Section 28

Conclusion

KasparAI's technical architecture demonstrates how a focused, full-stack application can deliver real business value through careful AI integration, graceful degradation, and clean separation of concerns. The system is built on well-established, production-ready technologies: React 19, Express 5, MongoDB, and OpenAI's GPT-4o-mini — each chosen for a specific reason rather than for novelty.

The behavioral tracking system, friction scoring engine, hesitation classifier, and multi-turn conversational recovery pipeline form a coherent, end-to-end data flow from browser event to AI conversation to recovered sale. The rule-based fallback ensures the product is fully functional without any external dependencies, while the AI layer unlocks a level of contextual precision that deterministic rules alone cannot achieve.

The most critical next steps for production readiness are: resolving the merge conflicts in conversation.js and shopify.js, wiring the dashboard to live analytics data, integrating Stripe for real payment processing, and adding authentication and rate limiting to the API. With those foundations in place, KasparAI is architecturally ready to scale from a hackathon demo to a production SaaS tool serving thousands of merchants.

Team	Track	Repository	Submission
Seven Up	Track 2	github.com/aadyasengar/checkout-intelligence-agent	KasparAI — Checkout Intelligence Agent